Deep Neural Network Compression & CV

End Term Project Report

Submitted by

Abhinav Saxena
Eragandula Nehasri
Hrishita Singh
Raghav Dangayach
Samridhi Jain
Sarthak Patel
Shriya Suravarapu
T. Krishnapriya Reddy
Utkarsh Kashyap

Contents

1 Introduction **2**

1.1 Objective 2

1.2 Project Overview 2

2 Model Architecture & Dataset **2**

2.1 Dataset 2

2.2 Feature Extraction 3

2.3 Classifier Architecture 3

3 Compression Methodology (The Pipeline) **4**

3.1 Baseline Training 4

3.2 Magnitude Pruning 5

3.3 K-Means Quantization 6

3.4 Huffman Encoding 6

3.5 Model Serialization 7

4 Experimental Results **8**

4.1 Accuracy Before and After 8

4.2 Storage Memory & Compression Ratio 8

4.3 Runtime Memory Impact 9

4.4 Discussion on CPU-only Training 9

5 Conclusion **10**

1. Introduction

1.1 Objective

The objective of this project is to reduce the storage cost and runtime memory footprint of a neural network while retaining as much classification accuracy as possible. The central idea is to apply a sequence of compression techniques to the trainable part of the model so that the final network occupies much less space than the original version.

1.2 Project Overview

In this work, we train a multilayer perceptron (MLP) classifier on CIFAR-10 image features extracted using a frozen pre-trained AlexNet backbone. After obtaining a baseline model, we compress it in a multi-stage pipeline consisting of:

1. magnitude-based pruning,
2. k-means weight quantization, and
3. Huffman encoding of the quantized weight indices.

Finally, the compressed model is serialized into a compact `.npz` file containing only the information necessary for reconstruction, namely sparse masks, cluster centers, cluster indices, and biases. This allows substantial reduction in storage compared to saving the entire dense model in standard format.

The purpose of this project is not only to demonstrate the implementation of compression techniques, but also to study the practical trade-off between model compactness and accuracy.

2. Model Architecture & Dataset

2.1 Dataset

The dataset used in this project is CIFAR-10, which contains 10 object categories. Since AlexNet expects larger input images, CIFAR-10 images are resized before feature extraction.

For training, data augmentation is applied to improve generalization. The augmentation pipeline includes:

- resizing the image to 256×256,
- random cropping to 224×224,

- random horizontal flipping,
- conversion to tensor, and
- normalization using CIFAR-10 mean and standard deviation.

For testing, the images are resized to 224×224 and normalized, but no random augmentation is applied.

2.2 Feature Extraction

Instead of training a full convolutional network from scratch, this project uses a pre-trained AlexNet as a frozen feature extractor. Only the `features` portion of AlexNet is used, and all of its parameters are kept fixed during training.

Each input image is passed through the frozen AlexNet feature extractor and the resulting feature map is flattened into a vector of size 9216. These 9216-dimensional vectors are then used as the inputs to the trainable classifier.

This design separates feature extraction from classification:

- the frozen AlexNet provides strong generic visual features,
- the MLP learns the final mapping from these features to CIFAR-10 classes.

2.3 Classifier Architecture

The classifier used in this project is a custom `CompressionMLP`. It is built using special `ModifiedLinear` layers that support pruning and quantization.

The classifier architecture is:

$$9216 \to 4096 \to 1024 \to 10$$

with ReLU activations and dropout layers in between.

More specifically, the network consists of:

- a dropout layer,
- a `ModifiedLinear` layer from 9216 to 4096,
- a ReLU activation,
- a second dropout layer,
- a `ModifiedLinear` layer from 4096 to 1024,

- another ReLU activation,
- a final `ModifiedLinear` layer from 1024 to 10.

The custom `ModifiedLinear` layer is important because it stores:

- a pruning mask,
- quantization cluster centers,
- quantized cluster indices,
- optional Huffman encoded bit streams and codebooks.

Thus, the same classifier can operate in normal, pruned, or quantized mode.

3. Compression Methodology (The Pipeline)

3.1 Baseline Training

The first stage of the pipeline is baseline training. The MLP classifier is trained on top of the frozen AlexNet features using cross-entropy loss and the Adam optimizer with learning rate 10^{-4}.

The baseline model is trained for 5 epochs. During each epoch:

1. images are passed through the frozen AlexNet feature extractor,
2. the extracted features are flattened,
3. the MLP classifier predicts class scores,
4. cross-entropy loss is computed,
5. gradients are backpropagated only through the MLP.

This stage provides the reference accuracy before compression.

```
@Simi1305 ➜ /workspaces/compressed_models (main) $ python "main (2).py"
CIFAR-10 loaded: 500 train, 100 test samples

--- Training Baseline MLP ---
Loading frozen feature extractor (AlexNet)...
/usr/local/python/3.12.1/lib/python3.12/site-packages/torch/utils/data/dataloader.py:775:
 UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then devi
ce pinned memory won't be used.
  super().__init__(loader)
Epoch [1/5] Average Loss: 2.2507
Epoch [2/5] Average Loss: 1.9781
Epoch [3/5] Average Loss: 1.6656
Epoch [4/5] Average Loss: 1.2838
Epoch [5/5] Average Loss: 1.0150
Evaluation Accuracy: 65.00%
```

Figure 1: Baseline training log showing the training loss over 5 epochs and the baseline evaluation accuracy.

3.2 Magnitude Pruning

After baseline training, magnitude-based pruning is applied to the weights of the `ModifiedLinear` layers.

The pruning method used is L1 magnitude pruning:

- all absolute weight values from the trainable linear layers are collected,
- a global threshold is computed using the required prune ratio,
- weights with magnitude below the threshold are masked to zero.

In this project, the prune ratio is 90%, which means approximately 90% of the weights are forced to zero. The weights are not physically deleted from the tensor during computation, but a binary mask is maintained so that the pruned weights do not contribute during the forward pass.

After pruning, the model is fine-tuned for 2 epochs so that the remaining weights can partially recover the lost accuracy.

```
--- Applying Pruning (90%) ---
Pruned: 37764223/41953280 (90.01% sparsity)
Fine-tuning pruned model...
Loading frozen feature extractor (AlexNet)...
Epoch [1/2] Average Loss: 1.1138
Epoch [2/2] Average Loss: 1.0129
Evaluation Accuracy: 64.00%
```

Figure 2: Pruning stage showing approximately 90% sparsity, followed by short fine-tuning and post-pruning evaluation accuracy.

3.3 K-Means Quantization

Once the model has been pruned and fine-tuned, the remaining non-zero weights are quantized using k-means clustering.

The idea of quantization is to replace many distinct floating-point weight values with a smaller number of representative cluster centers. In this project:

- only non-zero weights are considered,
- k-means clustering is applied with $k = 16$ clusters,
- each surviving weight is replaced by the nearest cluster center,
- instead of storing full 32-bit floating-point values, the model stores cluster indices and the 16 cluster centers.

Because $k = 16$, each cluster index can be represented using very few bits compared to a full float32 value. This gives a major reduction in memory usage.

```
--- Applying Quantization (16 clusters) ---
Quantized classifier.1 to 16 clusters.
Quantized classifier.4 to 16 clusters.
Quantized classifier.6 to 16 clusters.
Quantization complete. 3 layers quantized.
```

Figure 3: Quantization stage where the trainable linear layers are quantized into 16 clusters.

3.4 Huffman Encoding

After quantization, Huffman encoding is applied to the quantized cluster indices.

Even after replacing weights with cluster indices, some indices appear much more frequently than others. Huffman coding exploits this non-uniform frequency distribution by assigning:

- shorter bit codes to more frequent indices,
- longer bit codes to less frequent indices.

This reduces the average number of bits needed per stored index. The encoding is applied to the quantized non-zero indices of each layer.

As a result, the average bits per weight become significantly smaller than naive fixed-length encoding. This produces an additional compression gain on top of pruning and quantization.

```
--- Applying Huffman Encoding ---
Huffman encoded classifier.1: 3.88 bits/weight
Huffman encoded classifier.4: 3.97 bits/weight
Huffman encoded classifier.6: 4.00 bits/weight

Huffman encoding complete: 8.17x compression on indices
```

Figure 4: Huffman encoding statistics showing the average bits per weight for the quantized layers and the overall compression obtained on the indices.

3.5 Model Serialization

The final compressed model is saved in a compact `.npz` file rather than a standard dense PyTorch checkpoint.

Instead of storing every dense tensor directly, the serialization stores only the essential compressed representation:

- biases,
- sparse masks,
- quantization cluster centers,
- quantized cluster indices stored in compact integer form,
- optionally sparse weights for layers not yet quantized.

This is efficient because:

- boolean masks require far less space than dense float tensors,
- cluster indices can be stored as `uint8` for small numbers of clusters,
- cluster centers are few in number,
- the entire file is written using compressed NumPy serialization.

Therefore, the saved file is much smaller than the original model representation.

4. Experimental Results

4.1 Accuracy Before and After

The baseline model achieved an evaluation accuracy of 65.00% after 5 training epochs. After applying pruning to approximately 90% sparsity and then fine-tuning for 2 additional epochs, the model achieved an evaluation accuracy of 64.00%.

This shows that even after a very aggressive pruning step, the drop in accuracy is only about 1 percentage point for the reported run. Thus, a large reduction in parameter count was achieved while preserving most of the baseline classification performance.

Table 1: Accuracy comparison before and after compression stages

Stage	**Accuracy**
Baseline MLP after 5 epochs	65.00%
After 90% pruning + 2 epochs fine-tuning	64.00%

4.2 Storage Memory & Compression Ratio

The compression summary reported the following values:

- original model size: 228.00 MB,
- compressed size: 17.96 MB,
- compression ratio: 12.70×.

This is a substantial reduction in model size. In other words, the compressed model occupies less than one-sixteenth of the original storage while maintaining nearly the same accuracy as the baseline run.

```
============================================================
COMPRESSION SUMMARY
============================================================
Original model size: 228.00 MB
Compressed size: 17.96 MB
Compression ratio: 12.70x
============================================================
```

Figure 5: Compression summary showing original size, compressed size, and final compression ratio.

Table 2: Storage comparison

Metric	Value
Original model size	228.00 MB
Compressed model size	17.96 MB
Compression ratio	12.70×

4.3 Runtime Memory Impact

Compression also affects runtime memory usage, not just storage on disk.

In the uncompressed model, weights are typically stored as dense 32-bit floating-point tensors. In the compressed model:

- many weights are removed by pruning,
- non-zero weights are represented indirectly through compact cluster indices,
- the mask is stored separately,
- only a small set of cluster centers is needed to reconstruct the quantized weights.

Thus, the compressed representation reduces the amount of information that must be stored in memory. This is particularly valuable in memory-constrained settings, such as edge devices or systems with limited RAM.

Although reconstruction of weights may introduce some overhead at inference time depending on the implementation, the storage and load-time memory footprint are significantly reduced compared to a fully dense float32 model.

4.4 Discussion on CPU-only Training

One practical limitation in this project was the absence of GPU acceleration during experimentation. Since no GPU was available, the reported runs were executed on CPU. This made training and post-compression fine-tuning slower, which restricted the number of experiments, the amount of tuning, and the total training time that could be afforded.

As a result, the reported accuracy should be interpreted in that context. With GPU access, it would have been practical to:

- train for more epochs,
- perform more extensive hyperparameter tuning,
- fine-tune after quantization as well,

- test larger or repeated experimental runs.

Therefore, the modest accuracy values in the current report are justified as outcomes of a CPU-only implementation setting rather than a limitation of the compression pipeline itself. The main purpose here was to validate the end-to-end compression methodology and demonstrate meaningful compression with limited computational resources.

5. Conclusion

In this project, an MLP classifier trained on frozen AlexNet features for CIFAR-10 was successfully compressed using a three-stage pipeline: magnitude pruning, k-means quantization, and Huffman encoding.

The results show that:

- the baseline accuracy of 65.00% was largely preserved after compression,
- about 90% sparsity was introduced through pruning,
- quantization reduced the number of unique stored weight values to 16 cluster centers per layer,
- Huffman encoding further reduced the average bits needed to store the quantized indices,
- the final compressed representation reduced the model size from 228.00 MB to 17.96 MB,
- the overall compression ratio achieved was 12.70×.

Hence, the project demonstrates that substantial compression can be achieved without a major loss in predictive performance. This highlights the practical usefulness of combining pruning, quantization, and entropy coding for efficient neural network deployment.

References to Implementation

The implementation used a frozen AlexNet feature extractor, CIFAR-10 preprocessing with augmentation, a `CompressionMLP` based on `ModifiedLinear` layers, global pruning, k-means quantization, Huffman encoding, and compact `.npz` serialization.